

Release Notes — epl-20260504

Date: May 4, 2026

Base: upstream/develop

1. New Features

1.1 NDcPP Second Remote Admin (recovery-admin)

A second administrative account, `recovery`, is added alongside `admin` / `user` to satisfy NDcPP FIA_AFL.1 (Authentication Failure Handling).

The recovery account is intentionally limited — it can only manage account / lockout state, with NO privilege to change any other CLI setting.

Commit	Description
recovery-admin: foundation + per-account command tree	Per-account password / lockout policy persisted in <code>/etc/scli.yml</code> , per-account <code>set/show {status, policy, passwd, unlock}</code> command tree, and <code>pkg/auth/faillock.go</code> that parses live <code>faillock</code> output and projects it against the configured policy for status display.
recovery-admin: per-account <code>pam_faillock</code> template	<code>/etc/pam.d/common-auth</code> is now generated as a per-user faillock chain so each account (<code>admin</code> / <code>recovery</code> / <code>user</code> / <code>root</code>) carries its own <code>deny=</code> / <code>unlock_time=</code> values driven by <code>/etc/scli.yml</code> .
recovery-admin: cmdguard whitelist + tree-wide permission gates	New cmdguard strict-whitelist phase: at session start, every cobra leaf without an explicit <code>admin-or-recovery</code> / <code>recovery</code> / <code>all</code> annotation is pruned for the recovery user. Annotations applied across the tree so the security subtree, all system children except <code>account</code> , and state-modifying network commands are invisible to recovery.
recovery-admin: 'system account reset admin-passwd' command	recovery-only helper that generates a one-time random password for <code>admin</code> , prints it once, and back-dates the admin aging record so the next admin login is forced into a password change. y/N confirmation + audit log.

Initial credentials

Both accounts ship with self-documenting initial passwords. They are forced-expired on first login (NIAP FIA_PMG_EXT.1) so the operator must change them before the SCLI shell starts.

Account	Initial password
admin	admin
recovery	recovery

Logging in as recovery

```
1 ssh recovery@<device>
2 recovery@<device>'s password: recovery
3 Your password has expired. You must change it now.
4 Enter new password:
5 Re-enter new password:
6 Password for account 'recovery' updated successfully.
7 Welcome to the restricted shell! (Press TAB for auto-completion)
8 >>
```

Commands available to the recovery account

After login, only the following branches are visible — every other command is pruned by cmdguard:

Command	Notes
date	Show device date/time
exit	Leave the SCLI shell
system account show {admin recovery user} status	Includes faillock state (see below)
system account show {admin recovery user} policy	Per-account password / lockout policy
system account set admin unlock	Clear admin's faillock counter
system account set admin policy <field> <value>	Tune admin's password / lockout policy
system account set recovery passwd	Change recovery's own password
system account reset admin-passwd	recovery-only escape hatch — see §1.2

Typing `system <TAB>` from the recovery prompt returns only `account ;`
typing `security <TAB>` is rejected outright.

system account show <account> status output

Now includes a `Login failures: (or Login lockout: )` line driven by live `faillock` data and the configured `login-max-attempts / login-lockout-time` policy:

```
1 admin account: enabled
2 Password expiry: in 90 day(s)
3 Login failures: 0/3
```

After two bad password attempts:

```
1 admin account: enabled
2 Password expiry: in 90 day(s)
3 Login failures: 2/3 (1 attempt remaining)
```

When admin is locked out:

```
1 admin account: locked
2 Password expiry: in 90 day(s)
3 Login lockout: locked until 2026-05-04 09:23:11 (8m 24s remaining)
```

system account reset admin-passwd (recovery only)

The classic NDcPP recovery flow — admin lost the password, the device is locked out — without dragging in a console / serial recovery image. recovery types the command, confirms `y`, and the SCLI prints a random 16-character password ONCE on stdout. The admin aging record is back-dated (`password-max-age + 1` days) so the next admin login lands in expired state and must change the password before the shell starts.

```
1 recovery@>> system account reset admin-passwd
2 Are you sure you want to reset the admin password? [y/N]: y
3 Password for account 'admin' updated successfully.
4
5 =====
6 Admin password reset. Temporary password:
7     cjYi-Ssgw8KbdkRD
8 Admin must change this password at first login.
9 This password will not be shown again.
10 =====
11
12 >>
```

1.2 VPN Certificate Revocation Policy (vpn-revocation)

Adds RFC-5280 revocation policy knobs to libreswan and reshapes the SCLI VPN command tree to group revocation under its own subtree. The strongswan engine path stays intact but explicitly reports that the strict-mode knobs are libreswan-only.

Commit	Description
vpn: revocation knobs (ocsp/crl strict + interval) + 'set certs' subtree	Two related changes that together reshape security vpn set for PKI-based connections — see commands below.
akita-pki: seed NSS DB leaf cert during VPN client import	Libreswan with PKCS11-stored client cert needs the leaf cert in the NSS DB to fetch the CRL CDP at IKE_AUTH time. import-vpn-client-nss.sh now seeds the leaf alongside the existing key-cert PKCS11 import (idempotent — certutil deduplicates by nickname). Without this, security vpn save on libreswan stalled at "fetching CRL ..." on the first connect after EST enrollment.

Revocation in IKEv2

Two separate questions during IKE_AUTH determine whether a peer certificate is acceptable:

1. **Validation** — was the peer's certificate revoked? Answered by OCSP (live query to a responder) or CRL (cached blocklist).
2. **Strictness** — if validation cannot be reached (responder down, CRL stale, network partition), do we still allow the connection?

This release exposes both as user-controllable policy knobs. The defaults match NIAP guidance: validation enabled, strict mode on, periodic CRL refresh.

Command tree changes

Before (deprecated, removed):

```

1 security vpn set ocsp-validation {enable|disable}
2 security vpn set/del pubkey local-id <id>
3 security vpn set/del pubkey remote-id <id>
```

After:

```

1 security vpn set revocation ocsp-validation {enable|disable}
2 security vpn set revocation ocsp-strict {enable|disable} # libreswan only
3 security vpn set revocation crl-strict {enable|disable} # libreswan only
4 security vpn set revocation crl-check-interval <seconds 60..86400> # libreswan only
5
6 security vpn set/del certs local-id <id>
7 security vpn set/del certs remote-id <id>
```

The set/del pubkey subtree is removed in its entirety; both IDs now live under set certs next to ca-cert / key-cert / re-enroll, since they are only meaningful for certificate-based IKE_AUTH.

Per-knob behaviour

Knob	Default	Engine	Effect
revocation ocsf-validation enable	enabled	both	Send OCSF requests for peer certs at IKE_AUTH time
revocation ocsf-validation disable	—	both	Skip OCSF queries entirely
revocation ocsf-strict enable	enabled	libreswan	Soft-fail mode off — unreachable OCSF responder fails the connection
revocation ocsf-strict disable	—	libreswan	Soft-fail mode on — unreachable OCSF responder accepts the connection
revocation crl-strict enable	enabled	libreswan	CRL must be fresh and reachable; otherwise reject
revocation crl-strict disable	—	libreswan	Stale or unreachable CRL is tolerated
revocation crl-check-interval <s>	3600	libreswan	How often libreswan re-fetches CRLs from the CDP (60..86400 s)

On strongswan the SET path explicitly prints `Not supported on strongswan`; this setting only applies to libreswan. for the three libreswan-only knobs, and `vpn show config` omits them from the Revocation section entirely (only OCSF Validation is shown).

strongswan: OCSF + CRL handling (revocation plugin)

strongswan's revocation plugin runs OCSF and CRL as a **single verification chain**. Unlike libreswan, there is no per-method strict knob — strongswan exposes one connection-level policy instead.

The shipped default is **revocation = ifuri**, which means:

- If the peer cert carries an OCSF URI (AIA) or a CRL URI (CDP)
→ revocation MUST be checked, and an unreachable / inconclusive result FAILS the SA.
- If the peer cert carries neither URI → revocation is skipped and the cert is accepted (nothing to query).

Verification order (when ocsf-validation = yes)

1. OCSF first.

- Read the OCSF responder URL from the peer cert's AIA (Authority Information Access) extension and send a live status request.
- good → accept and stop.

- revoked → reject the SA immediately.
- unknown or transport failure → fall through to CRL.

2. CRL fallback.

- Download the CRL from the URL listed in the peer cert's CDP (CRL Distribution Point) extension and check the cert's serial number against the revocation list.
- CRLs are cached until their `nextUpdate` field expires; the next check refreshes automatically.

3. Final decision (`ifuri` policy).

- If a URI was present (either AIA or CDP) but verification failed or returned inconclusive → reject the SA.
- If neither URI was present → accept (no source to query).

What the operator sees

There is no separate `set revocation crl-strict / crl-check-interval` on strongswan because the `ifuri` policy folds those decisions into the per-cert URI presence check. The only operator knob on strongswan is the master toggle:

- `security vpn set revocation ocsdp-validation enable` (default) — run the chain above for every peer cert.
- `security vpn set revocation ocsdp-validation disable` — skip the entire chain (no OCSP, no CRL, no rejection); intended for lab / debugging only.

security vpn show config — new Revocation section

```

1  Revocation:
2      OCSP Validation      : enabled
3      OCSP Strict         : enabled      (libreswan only)
4      CRL Strict          : enabled      (libreswan only)
5      CRL Check Interval  : 3600 s      (libreswan only)
```

On strongswan:

```

1  Revocation:
2      OCSP Validation      : enabled
```

Where it lands on disk (libreswan)

`security vpn save` writes the three values into `/etc/ipsec.conf` under the connection block:

```

1  conn secure-network
2      ...
3      ocsdp-strict=yes
4      crl-strict=yes
5      crlcheckinterval=3600
```

The strongswan path leaves these out of `/etc/swanctl/swanctl.conf` since strongswan has no equivalent settings.

NSS DB leaf cert seeding (libreswan)

When `security vpn save` provisions a libreswan client cert from the TPM2 PKCS#11 token, the leaf cert is now also imported into NSS (`/etc/ipsec.d`). Libreswan reads the private key from the token but needs the leaf in NSS to follow the CRL distribution point URL during `IKE_AUTH`; without this seed step, the first connect after EST enrollment stalled at `fetching CRL . . . .`

No user-visible CLI change — the import is automatic and idempotent.

2. Migration Notes

2.1 Existing scripts using deprecated VPN commands

Update any provisioning script that calls the old flat command paths:

Old	New
<code>security vpn set ocsp-validation enable</code>	<code>security vpn set revocation ocsp-validation enable</code>
<code>security vpn set pubkey local-id <id></code>	<code>security vpn set certs local-id <id></code>
<code>security vpn set pubkey remote-id <id></code>	<code>security vpn set certs remote-id <id></code>
<code>security vpn del pubkey local-id</code>	<code>security vpn del certs local-id</code>
<code>security vpn del pubkey remote-id</code>	<code>security vpn del certs remote-id</code>